

Aula 29 - PHP Orientado a Objetos I

Descrição: Exploração técnica de estruturas de dados coletivas (arrays) e introdução aos pilares da POO: classes, atributos e métodos. Criação da classe modelo (entidade) para o projeto da Situação de Aprendizagem.

Pergunta:

"Antes de começarmos, pensem: se vocês tivessem que gerenciar as informações de 10.000 produtos diferentes em um array manual, como garantiriam que nenhum desenvolvedor acidentalmente cadastrar um 'preço' como sendo um texto em vez de um número?"

Arrays e Coleções de Dados no PHP

A. CONCEITO

No PHP, os arrays são ferramentas extremamente versáteis. Eles não são apenas listas; são mapas ordenados que podem assumir diferentes formas dependendo da sua necessidade de organização.

Tipos de Arrays

Entender arrays no PHP é como aprender a organizar diferentes tipos de armários. Dependendo do que você quer guardar e de como quer encontrar depois, você escolhe uma estratégia diferente.

Antes de mergulharmos na prática, precisamos entender as três faces dos arrays:

1. Arrays Indexados (A Lista de Chamada)

É o tipo mais comum. Imagine uma fila ou uma lista de compras. A posição de cada item é definida por um número (índice), e no PHP, essa contagem **sempre começa no 0**.

- **Como funciona:** Você foca nos valores. O PHP se encarrega de dar um "número de identificação" para cada um na ordem em que aparecem.
- **Exemplo prático:**

PHP

```
$frutas = ["Maçã", "Banana", "Morango"];  
// O PHP entende: 0 => Maçã, 1 => Banana, 2 => Morango  
echo $frutas[1]; // Saída: Banana
```

2. Arrays Associativos (O Dicionário)

Aqui, em vez de números automáticos, você escolhe uma **palavra-chave (string)** para identificar o valor. É como uma agenda de contatos ou uma etiqueta de identificação.

- **Como funciona:** Você usa o operador `=>` para associar uma "Etiqueta" a um "Conteúdo". Isso torna o código muito mais fácil de ler (semântico).
- **Exemplo prático:**

PHP

```
$carro = [  
    "marca" => "Toyota",  
    "modelo" => "Corolla",  
    "ano" => 2024  
];  
echo $carro["modelo"]; // Saída: Corolla
```

3. Arrays Multidimensionais (A Planilha ou Matriz)

Imagine uma tabela do Excel. Você tem linhas e colunas. No PHP, um array multidimensional nada mais é do que um **array que guarda outros arrays** dentro dele.

- **Como funciona:** É usado para estruturas de dados complexas, onde um único item possui várias subinformações. Para acessar um dado, você usa dois ou mais colchetes (um para "entrar" no primeiro array e outro para o "interno").
- **Exemplo prático:**

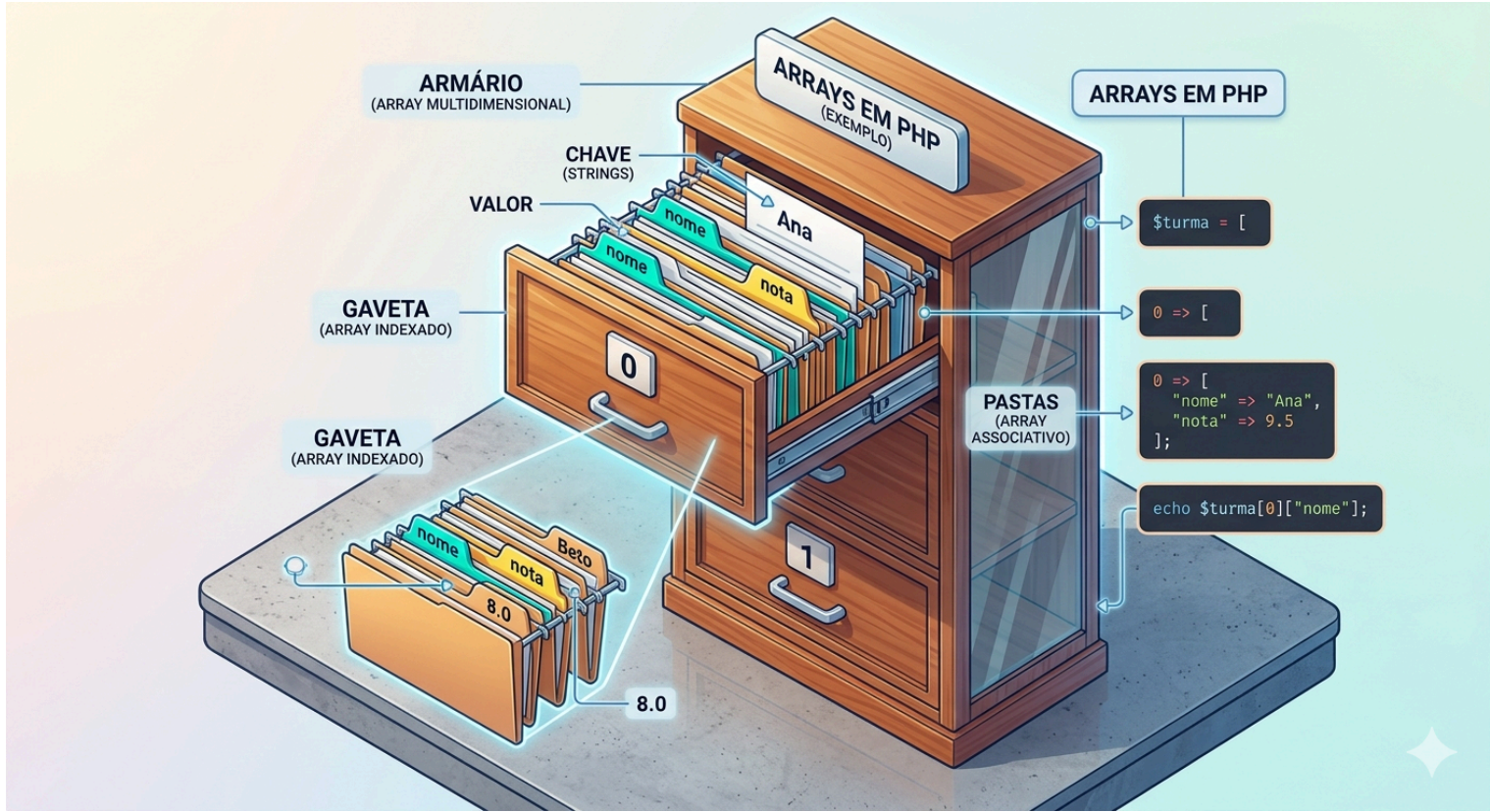
```
PHP
$usuarios = [
    ["nome" => "Ana", "email" => "ana@email.com"],
    ["nome" => "Beto", "email" => "beto@email.com"]
];

// Acessando o nome do segundo usuário (índice 1)
echo $usuarios[1]["nome"]; // Saída: Beto
```

Resumo Comparativo

Tipo	Chave (Key)	Uso Ideal
Indexado	Números (0, 1, 2...)	Listas simples, ordem cronológica.
Associativo	Strings ("nome", "preco")	Objetos, registros únicos, configurações.
Multidimensional	Números ou Strings	Tabelas, catálogos, sistemas de coordenadas.

B. VISUALIZANDO



C. EXEMPLO DE CÓDIGO

Para consolidar o que aprendemos, vamos montar um cenário real: um **Sistema de Gestão de Frota**. Neste exemplo, utilizaremos os três tipos de arrays para organizar as informações de maneira lógica.

PHP

```
<?php
```

```
// 1. ARRAY INDEXADO (Lista simples de categorias)
```

```
$categorias = ["Sedan", "SUV", "Hatch"];
```

```
// 2. ARRAY ASSOCIATIVO (Dados técnicos de um veículo específico)
```

```
$veiculo_destaque = [  
    "modelo" => "Corolla",  
    "marca" => "Toyota",  
    "ano" => 2024,  
    "cor" => "Prata"  
];
```

```
// 3. ARRAY MULTIDIMENSIONAL (Tabela/Matriz de vários veículos)
```

```
$frota = [  
    [  
        "placa" => "ABC-1234",  
        "status" => "Disponível",  
        "km" => 15000  
    ],  
    [  
        "placa" => "XYZ-5678",  
        "status" => "Em manutenção",  
        "km" => 42000  
    ]  
];
```

```
// --- EXIBINDO OS DADOS ---
```

```
echo "--- CATEGORIAS ---\n";
```

```
echo "A primeira categoria é: " . $categorias[0] . "\n\n";
```

```
echo "--- DESTAQUE ---\n";
```

```
echo "Veículo: " . $veiculo_destaque["modelo"] . " (" . $veiculo_destaque["marca"] . ")\n\n";
```

```
echo "--- RELATÓRIO DE FROTA ---\n";
```

```
// Acessando o status do segundo veículo (índice 1) da frota
```

```
echo "O veículo de placa " . $frota[1]["placa"] . " está " . $frota[1]["status"] . " .";
```

```
?>
```

D. EXPLICAÇÃO DO CÓDIGO

1. Array Indexado (\$categorias):

- O que aconteceu: Criamos uma lista onde a ordem importa.
- Acesso: O PHP atribuiu automaticamente o índice 0 para "Sedan", 1 para "SUV" e 2 para "Hatch". Para exibir "Sedan", usamos \$categorias[0].

2. Array Associativo (\$veiculo_destaque):

- O que aconteceu: Em vez de números, usamos etiquetas (chaves) que descrevem o dado.
- Acesso: Usamos o operador => para ligar, por exemplo, a chave "ano" ao valor 2024. Isso torna o código semântico: \$veiculo_destaque["modelo"] é muito mais fácil de entender do que um índice numérico.

3. Array Multidimensional (\$frota):

- O que aconteceu: Criamos uma "super variável" que armazena outros arrays. Cada item dentro de \$frota é, por si só, um array associativo.
- Acesso: Para chegar ao dado, usamos dois níveis de colchetes:
 - O primeiro [1] seleciona o segundo "bloco" de dados da lista (o segundo veículo).
 - O segundo ["placa"] entra dentro desse bloco para pegar apenas o valor da placa.

E. DICAS

1. **Aspas sempre:** Sempre utilize aspas (simples ou duplas) nas chaves de strings. O PHP moderno gera avisos de performance e segurança se você esquecer.
 2. **Cuidado com o Case:** Chaves são *case-sensitive*. "Nome" é diferente de "nome".
 3. **Sempre** utilize aspas simples ou duplas nas chaves de strings. Embora o PHP aceite chaves sem aspas em alguns contextos (**tratando-as como constantes**), isso gera avisos de performance e segurança em versões modernas.
-

Classes e Atributos: O Molde e suas Características

A. CONCEITO

A Programação Orientada a Objetos (POO) é uma filosofia de construção de sistemas que busca aproximar o código do mundo real por meio de objetos que carregam dados e comportamentos. Aqui está uma explicação didática dos conceitos **Classes** e **Atributos**.

Classe: O Projeto ou Molde

Uma **Classe** é a estrutura que define como um objeto será. Imagine que você quer fabricar vários carros de um mesmo modelo; a classe seria o **projeto de engenharia** ou a **planta baixa** desse carro.

- **Função:** Ela declara formalmente quais dados o objeto terá e como ele se comportará.
- **Declaração no PHP:** Utilizamos a palavra-chave `class` seguida do nome da classe.
- **Instanciação:** Para criar um objeto real a partir dessa planta, usamos o operador `new`. Esse processo cria uma "instância" (o objeto propriamente dito) que ocupará um espaço na memória.

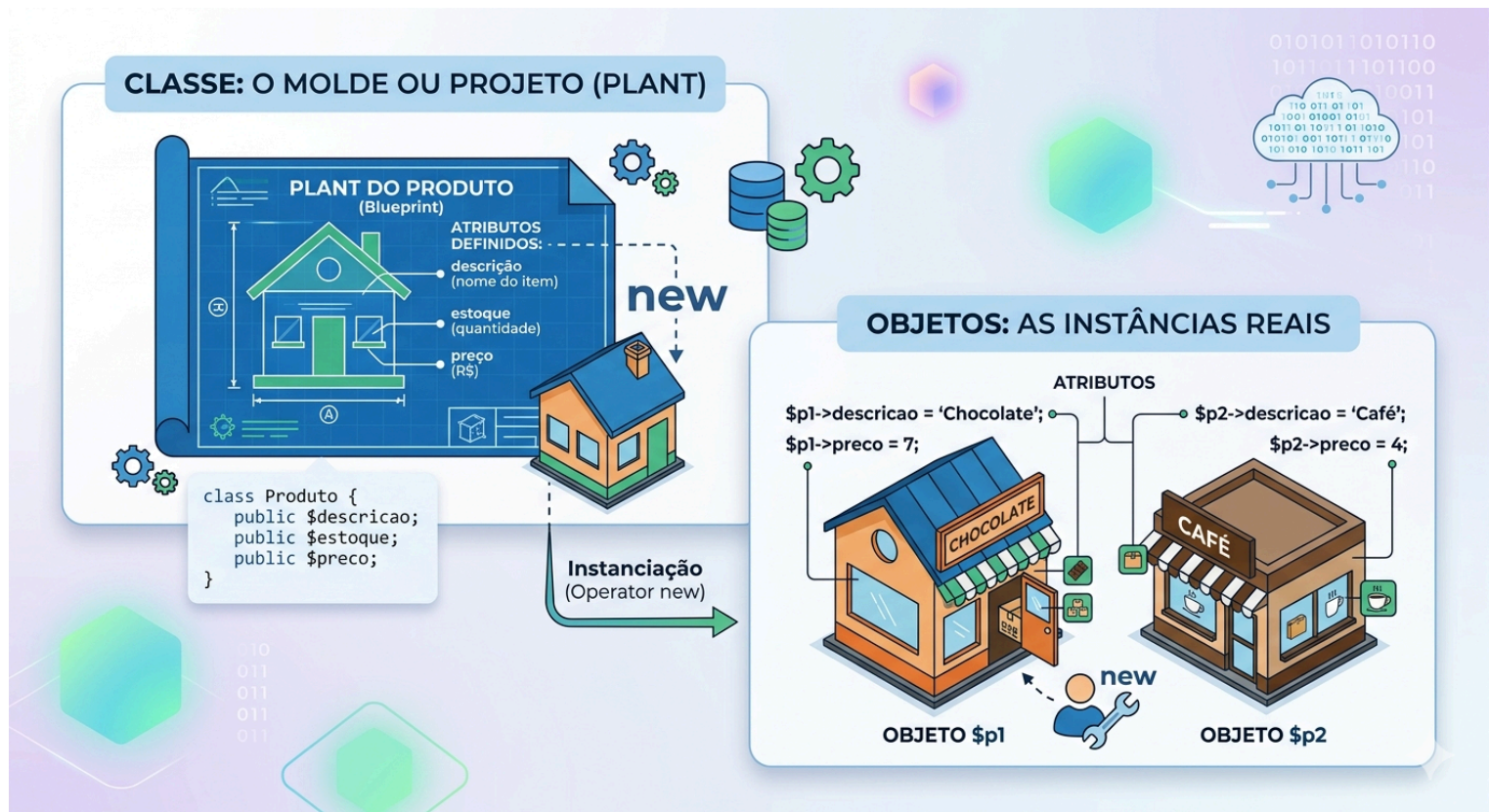
Atributos: As Características (Dados)

Os **Atributos** representam as características ou o estado de um objeto. Se a classe for um "Produto", os atributos seriam as informações que o descrevem.

- **Exemplos comuns:** Descrição, preço e estoque são atributos típicos de um objeto Produto.
- **Visibilidade:** No código, os atributos são definidos no início da classe usando palavras-chave como `public`, `protected` ou `private`.

Nota didática: Embora atributos públicos permitam acesso direto, o uso de atributos privados ou protegidos é preferível para garantir o **encapsulamento** (segurança dos dados).

B. VISUALIZANDO



C. EXEMPLO DE CÓDIGO

Abaixo, veja como transformamos a planta (Classe) em objetos reais com características próprias (Atributos):

PHP

<?php

// A CLASSE: O molde para todos os produtos

```
class Produto {  
    public $descricao; // Atributo  
    public $estoque; // Atributo  
    public $preco; // Atributo  
}
```

// OS OBJETOS: Instâncias reais criadas a partir do molde

```
$p1 = new Produto;  
$p1->descricao = 'Chocolate'; // Definindo o valor do atributo  
$p1->preco = 7;
```

```
$p2 = new Produto;  
$p2->descricao = 'Café'; // Cada objeto tem seus próprios atributos  
$p2->preco = 4;  
?>
```

Neste exemplo, \$p1 e \$p2 são objetos distintos que compartilham a mesma estrutura (Classe), mas possuem dados diferentes (Atributos).

D. EXPLICAÇÃO DO CÓDIGO

1. A Definição da Classe (class Produto)

- class Produto { ... }: Aqui você está batizando o seu modelo. Por convenção, nomes de classes em PHP começam com letra maiúscula.

- `public $descricao`;, `$estoque`;, `$preco`;: Estes são os Atributos. Eles representam as características que o objeto terá. A palavra `public` (visibilidade) indica que esses valores podem ser acessados e alterados de qualquer lugar do código.

2. Criação de Objetos (Instanciação)

- `$p1 = new Produto`;: O operador `new` cria um novo objeto na memória. Agora, `$p1` é uma instância real e única da classe `Produto`.
- `$p2 = new Produto`;: Criamos um segundo objeto. Embora usem o mesmo molde, `$p1` e `$p2` são independentes. Se você mudar o preço de um, o outro não será afetado.

3. Manipulação de Atributos (->)

O operador seta (->) é a ferramenta que o PHP usa para "entrar" no objeto e acessar seus componentes.

- `$p1->descricao = 'Chocolate'`;: Aqui você está dizendo: "No objeto `$p1`, procure o atributo `descricao` e atribua a ele o texto 'Chocolate'".
- Diferenciação: Note que no objeto `$p2`, a descrição é definida como `'Café'`. Isso demonstra o poder da POO: você tem uma única estrutura de código para gerenciar múltiplos itens com dados diferentes.

O que acontece com o `$estoque`?

No código, o atributo `$estoque` foi definido na classe, mas nunca recebeu um valor nos objetos. Por padrão, se você tentasse exibir `$p1->estoque` agora, ele estaria vazio (`null`), pois não foi inicializado.

E. ATENÇÃO!

Nunca deixe atributos sensíveis como `public`. Se qualquer parte do sistema puder alterar o `saldo` de uma conta bancária sem passar por uma validação, você criou uma falha de segurança gravíssima.

Métodos: Comportamentos e Acesso Seguro

A. CONCEITO

Os **métodos** são um dos pilares fundamentais da Programação Orientada a Objetos (POO). Se os atributos são as "características" de um objeto, os métodos são as suas **ações** ou **comportamentos**.

O que é um Método?

Um método é, essencialmente, uma função escrita dentro de uma classe. Ele representa o que os objetos daquela classe podem **fazer**.

- **Analogia**: Se pensarmos em um objeto da classe `Carro`, os atributos seriam "cor" e "modelo", enquanto os **métodos** seriam as ações "acelerar", "frear" e "ligar".

A Sintaxe no PHP

Para declarar um método, utilizamos a palavra-chave `function` dentro do corpo da classe. Assim como as funções comuns, eles podem receber parâmetros e retornar valores.

```
PHP
class Conta {
    public $saldo;

    // Um método para depositar valores
    public function depositar($valor) {
        if ($valor > 0) {
            $this->saldo += $valor; // Modifica um atributo do próprio objeto
        }
    }
}
```

- **O uso do \$this:** Dentro de um método, utilizamos a pseudo-variável \$this para referenciar atributos ou outros métodos do próprio objeto que está executando a ação.

Tipos Especiais de Métodos

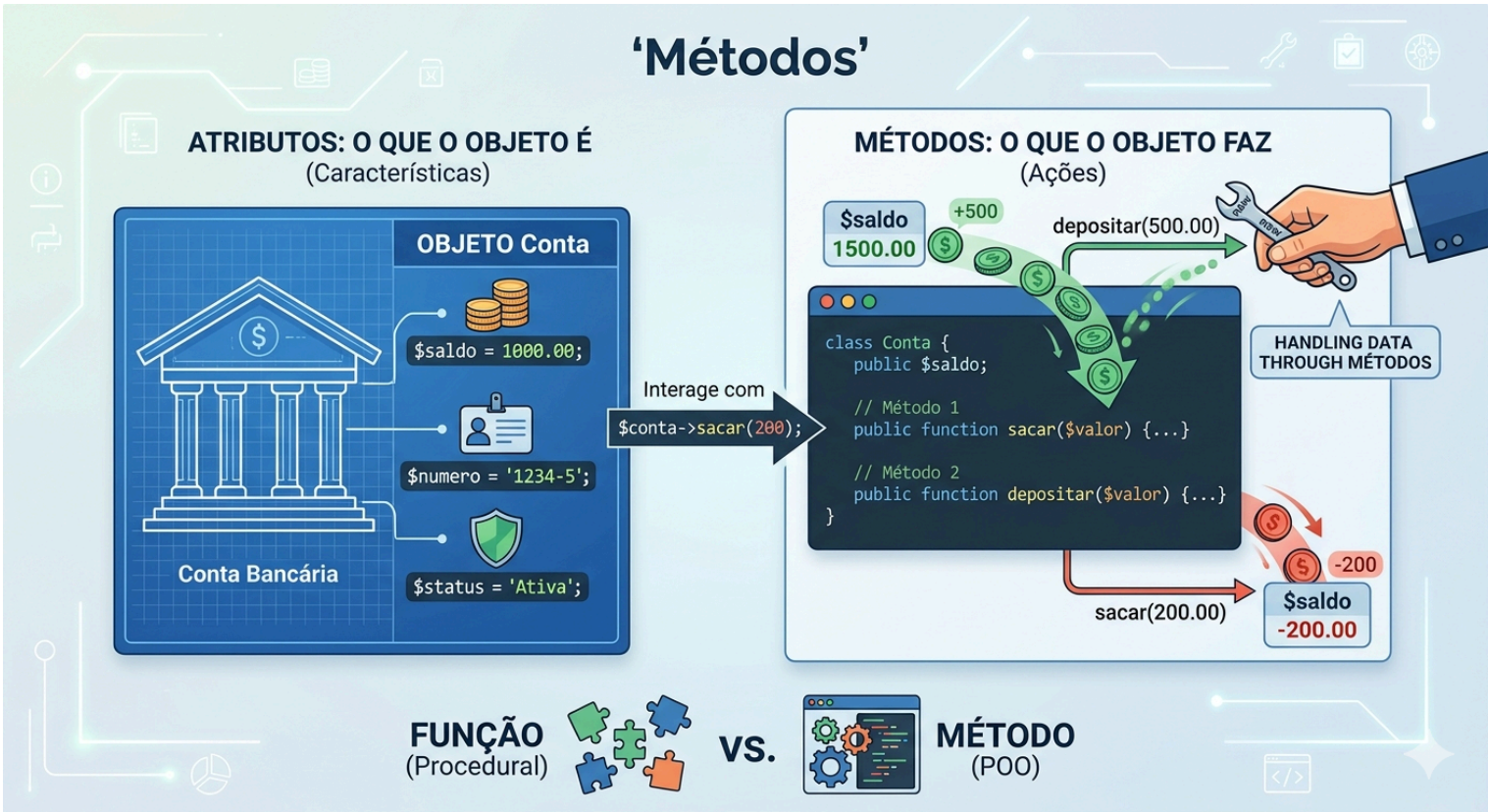
Destacamos categorias importantes de métodos que resolvem problemas específicos:

- **Métodos Construtores (__construct):** São executados automaticamente assim que um novo objeto é criado (instanciado). Eles servem para inicializar atributos ou realizar tarefas de configuração inicial.
- **Métodos Estáticos (static):** Diferente dos métodos comuns, os métodos estáticos pertencem à classe e não a um objeto específico. Eles podem ser chamados sem a necessidade de criar uma instância (usando Classe::metodo()).
- **Métodos Mágicos:** São métodos que o PHP executa automaticamente em situações especiais. Exemplos incluem o __get() e __set(), que interceptam tentativas de ler ou gravar em atributos, e o __toString(), que define como o objeto deve se comportar quando for exibido como texto.

Por que usar métodos?

1. **Encapsulamento:** Os métodos permitem controlar como os dados (atributos) são alterados. Em vez de mudar o saldo de uma conta diretamente, você chama o método depositar(), que pode validar se o valor é positivo.
2. **Reutilização:** Uma vez definido na classe pai, o comportamento pode ser herdado por outras classes (Herança), evitando a duplicação de código.
3. **Abstração:** Você não precisa saber como o método faz o cálculo interno; basta saber o que ele faz e quais dados ele precisa para funcionar.

B. VISUALIZANDO



C. EXEMPLO DE CÓDIGO

Para ilustrar o conceito de **Métodos**, vamos criar uma classe chamada `ContaBancaria`. Nela, veremos como os métodos controlam as ações (**comportamentos**) e como eles protegem e manipulam os dados (**atributos**).

PHP

```
<?php
```

```
class ContaBancaria {
    public $titular;
    public $saldo;

    // MÉTODO CONSTRUTOR: Executado automaticamente ao criar o objeto
    public function __construct($nome, $valorInicial) {
        $this->titular = $nome;
        $this->saldo = $valorInicial;
        echo "Conta de {$this->titular} criada com sucesso!\n";
    }

    // MÉTODO DE AÇÃO: Realiza um depósito
    public function depositar($valor) {
        if ($valor > 0) {
            $this->saldo += $valor;
            echo "Depósito de R$ {$valor} realizado.\n";
        }
    }

    // MÉTODO DE AÇÃO: Realiza um saque com validação
    public function sacar($valor) {
        if ($valor > 0 && $valor <= $this->saldo) {
            $this->saldo -= $valor;
            echo "Saque de R$ {$valor} autorizado.\n";
        } else {
            echo "Saque de R$ {$valor} negado: Saldo insuficiente.\n";
        }
    }

    // MÉTODO DE RETORNO: Exibe o saldo atual
    public function obterSaldo() {
        return "Saldo atual de {$this->titular}: R$ {$this->saldo}\n";
    }
}
```

```
// --- UTILIZANDO OS MÉTODOS ---
```

```
$minhaConta = new ContaBancaria("Maria", 500); // Chama o __construct
```

```
$minhaConta->depositar(200); // Executa a ação de depositar
```

```
$minhaConta->sacar(100); // Executa a ação de sacar
```

```
$minhaConta->sacar(1000); // Tentativa de saque maior que o saldo
```

```
echo $minhaConta->obterSaldo(); // Exibe o resultado final
```


D. EXPLICAÇÃO DO CÓDIGO

1. O Método Especial `__construct`

- Este é o "porteiro" da classe. Ele garante que, assim que você der um `new ContaBancaria`, o objeto já nasça com um nome e um saldo inicial. Sem ele, os atributos ficariam vazios até que alguém os preenchesse manualmente.

2. O uso do `$this`

- Observe que dentro dos métodos usamos `$this->saldo`. Isso é fundamental. O `$this` diz ao PHP: "Ei, mexa no saldo DESTA conta específica que está chamando o método agora". Sem o `$this`, o PHP procuraria uma variável local comum e o código falharia.

3. Métodos com Parâmetros (`$valor`)

- Os métodos `depositar` e `sacar` recebem informações externas através de variáveis entre parênteses. Isso permite que a mesma ação seja executada com valores diferentes a cada chamada.

4. Lógica e Validação (Encapsulamento)

- A grande vantagem do método sobre a alteração direta do atributo (ex: `$conta->saldo = -500`) é a **proteção**.
- No método `sacar`, incluímos um `if`. Ele impede que o saldo fique negativo. O método funciona como um filtro de segurança para os dados do objeto.

5. Métodos com Retorno (`return`)

- O método `obterSaldo` não apenas executa uma tarefa, ele "devolve" uma resposta para quem o chamou. Isso é útil para processar informações e depois exibi-las ou usá-las em outros cálculos.

E. REFLITA

Se você tem 50 telas no seu sistema que exibem o preço formatado e hoje o cliente decide mudar de "R\$" para "US\$", onde seria mais fácil alterar: em 50 arquivos HTML ou em um único método `exibirPrecoFormatado()` dentro da classe?

FAQ - Perguntas Frequentes

Pergunta: Posso criar uma classe sem o método `__construct`?

Resposta: Sim! O PHP criará um construtor vazio padrão. Porém, se o seu objeto precisa de dados básicos para "existir" (como o CPF de um cliente), é boa prática obrigar esses dados no construtor.

Pergunta: Qual a diferença real entre Array Associativo e Classe?

Resposta: O array é apenas um "saco de dados". Ele não tem comportamento. A Classe une os dados aos comportamentos (métodos). Use arrays para listas simples e Classes para representar entidades do seu negócio.

Pergunta: O que é o erro "Cannot access private property"?

Resposta: Isso acontece quando você tenta fazer algo como `$produto->preco = 10`; fora da classe. Como o atributo é `private`, você **deve** usar o método setter: `$produto->setPreco(10)`;

Pergunta: Posso usar IA para gerar Getters e Setters?

Resposta: Sim, IDEs e IAs são ótimas para isso (é um código repetitivo). No entanto, a **lógica de validação** (ex: não aceitar estoque negativo) é responsabilidade sua e deve ser revisada.

Pergunta: Por que usar tipagem (string, int) nos parâmetros?

Resposta: Para evitar erros bobos. Se você espera um número para o cálculo de imposto e recebe uma string "dez", o PHP 8 com `strict_types` avisará o erro na hora, em vez de fazer um cálculo errado silencioso.

RESUMINDO

Arrays e Coleções de Dados

- **Arrays Indexados:** Listas numeradas onde cada valor é acessado por um índice inteiro que começa em 0.
- **Arrays Associativos:** Estruturas tipo "dicionário" que utilizam chaves nominais (strings) para identificar valores de forma semântica.
- **Arrays Multidimensionais:** Conjuntos complexos onde um array armazena outros arrays, funcionando como tabelas ou matrizes.

Classes e Atributos

- **Classe:** O projeto técnico ou molde que define a estrutura e os comportamentos de um objeto.
- **Instanciação (new):** O processo de criar um objeto real e individual a partir de uma classe na memória.
- **Atributos:** Variáveis internas de uma classe que armazenam as características ou o estado de um objeto.
- **Visibilidade (public/private):** Regras que definem quem pode acessar ou alterar os dados internos de um objeto.

Métodos: Comportamentos e Acesso Seguro

- **Métodos:** Funções internas de uma classe que definem as ações e comportamentos que um objeto pode realizar.
 - **Pseudo-variável \$this:** Referência interna usada para que um método acesse atributos ou outros métodos do próprio objeto.
 - **Método Construtor (__construct):** Função executada automaticamente para configurar o objeto no exato momento da sua criação.
 - **Encapsulamento:** Prática de usar métodos para validar e proteger a alteração de atributos, garantindo a segurança dos dados.
 - **Abstração:** Capacidade de utilizar as funções de um objeto sem precisar conhecer a complexidade do código interno.
-

FECHAMENTO

Conexão com a SA: O que fizemos hoje é a base do seu **CRUD**. As classes que vocês criaram serão as "Entidades" que o banco de dados irá preencher. Sem essas classes, vocês teriam que tratar cada dado do banco manualmente em todas as páginas, o que tornaria o projeto final impossível de entregar no prazo.